Beyond Static Leaderboards: Drift-Aware, Bias-Controlled, and Cost-Normalized Evaluation of LLMs for Accessible Software Engineering

[TODO: Author name(s)]
[TODO: Affiliation, city, country]
hasebahmd.ai@gmail.com

***Abstract*—Large language models (LLMs) now write, repair and audit user-interface code, so the accessibility of what they produce decides who can use the software they help build. Yet models are compared on static leaderboards that ignore accessibility, and accessibility studies of LLMs typically test one or two models with one-shot prompts, sometimes let models judge their own outputs, and rarely report cost. We propose an evaluation framework for LLM-assisted accessible software engineering with four parts: (i) a six-dimension capability taxonomy that maps general LLM capabilities to WCAG 2.2-anchored tasks and oracles; (ii) the Dynamic Contextual Drift Index (DCDI), the share of accessibility adherence an agent loses over a long, tool-using session relative to a counterfactual fresh-context control; (iii) Cross-Model Blind Peer Evaluation (CMBPE), an anonymized, family-recused jury that judges every pair in both orders and adjusts for position and length bias, calibrated against ratings by blind and low-vision people; and (iv) Capability-per-Dollar-Second (CPDS), a scenario-weighted efficiency score with probabilistic Pareto analysis. We apply the framework to TBD frontier and open-weight models on accessible UI generation, seeded-fault repair and auditing, 24-step agentic builds and alt-text writing. [TODO: Two or three sentences with the headline results for RQ2 (drift), RQ3 (judging) and RQ4 (efficiency), using values from results/values.tex.] All tasks, prompts, code and logs are released.**

***Index Terms*—accessibility, large language models, LLM evaluation, LLM-as-a-judge, WCAG, agentic software engineering**

I. Introduction

Software increasingly mediates access to health care, public services and work, yet most of it remains inaccessible: in February 2026, automated tests detected WCAG 2 failures on 95.9% of the one million most popular home pages, with 56.1 errors per page on average [1]. Regulation is raising the stakes, with the European Accessibility Act applying since June 2025 [2], just as LLMs are increasingly used to write user interfaces. LLM-generated interfaces routinely contain accessibility barriers [3]–[6], but the same models can detect and repair barriers and describe images [7]–[9]. Which model a team chooses, and how it uses it, therefore shapes who can use the software it builds.

Choosing well requires evaluation that reflects how models are used. General comparisons rely on static leaderboards: crowd-sourced Bradley–Terry ratings [10], knowledge benchmarks such as MMLU-Pro [11], and functional coding benchmarks such as HumanEval and SWE-bench [12], [13]. None measures accessibility. Accessibility-specific studies fill part of the gap but share three blind spots. First, they test one-shot prompts or loops of at most three iterations [8], [14], whereas coding assistants now run long, tool-using sessions in which early instructions are known to fade [15], [16]. Second, automated checkers cover only part of WCAG [17], so semantic quality (is this alt text *useful*?) is increasingly judged by LLMs, sometimes by the very models being judged [18], despite documented position, verbosity and self-preference biases [19]–[21]. Third, cost and latency are rarely reported (one exception studies a single model [8]), although affordability decides whether under-resourced public bodies, non-profits and small teams can use AI-assisted remediation at all.

We address these gaps with an evaluation framework and a pre-registered study of current frontier and open-weight models. We ask: **RQ1** How do models perform across six accessibility-engineering capabilities? **RQ2** Do agents keep honoring accessibility requirements as a session grows? **RQ3** Can a blind, cross-family jury judge accessibility content more reliably than a single LLM judge? **RQ4** Which models are efficient in capability, cost and latency? Our contributions are:

- *Dynamic Contextual Drift Index (DCDI)*: the share of accessibility adherence an agent loses over a long, tool-using session relative to a counterfactual fresh-context control with the identical project state, plus a padded control that separates the effect of context length from that of accumulated history.
- *Cross-Model Blind Peer Evaluation (CMBPE)*: a judging protocol that anonymizes outputs, recuses judges from their own model family, presents every pair in both orders, and aggregates verdicts with a Bradley–Terry model that estimates each judge's position and length bias; we validate it against blind and low-vision readers' ratings.
- *Capability-per-Dollar-Second (CPDS)*: a scenario-weighted efficiency score built from per-task cost and end-to-end latency, with bootstrap Pareto-membership probabilities.

These rest on a capability taxonomy that maps general LLM capabilities to WCAG 2.2-anchored tasks and oracles (Section III). [TODO: One or two sentences previewing the main findings.]

II. Background and Related Work

Static and dynamic LLM evaluation. Chatbot Arena ranks models by fitting a Bradley–Terry model [22] to crowd preferences [10]; MMLU-Pro, HumanEval and SWE-bench score single answers or patches against fixed references [11]–[13]; HELM adds efficiency and robustness to accuracy [23]. Static suites are prone to contamination [24] and say little about extended interaction: models miss information in the middle of long contexts [16], [25] and perform 39% worse on average when a task unfolds over several turns [15], while agent benchmarks measure task success over multi-turn tool use [26]. DCDI differs in what it tracks, adherence to a *non-functional* requirement stated once, and in how it isolates drift, through a paired counterfactual control.

LLMs as judges. G-Eval popularized rubric-based LLM scoring [27]. LLM judges favor the first position, longer answers and their own outputs [19]–[21]; panels of diverse judges reduce intra-model bias [28], and regressing out length reduces verbosity bias [29]. In software engineering, LLMs can replace some manual annotation but need validation [30], and guidelines call for human validation and open-model baselines [31], [32]. CMBPE combines recusal, order permutation and bias-aware aggregation, calibrated against the people the content is for.

Cost-aware evaluation. Agents should be compared on joint cost–accuracy Pareto frontiers [33], and cost-of-pass expresses accuracy as the expected cost of a correct answer [34]. CPDS adds latency, deployment scenarios and uncertainty in frontier membership.

AI and accessibility in software engineering. Software-engineering research has studied accessibility defects and their repair in mobile apps [35], [36] and practitioners' views of accessibility [37]. For LLMs, generated web and mobile UIs contain frequent violations [3]–[6]; checker feedback helps more than prompting techniques [14]; assistant design shapes novices' practice [38]; and LLM judges find semantic faults that automated checks miss [18]. LLM-based repair resolves many syntactic violations but also introduces regressions [7], [8], [39]. For images, what a description should say depends on context and on blind and low-vision (BLV) readers' needs [9], [40]–[42]. A recent review finds few shared benchmarks and limited involvement of disabled people [43]. To our knowledge, no prior study combines many current models, long agentic sessions, bias-controlled judging and cost-aware comparison for accessibility.

III. A Capability Taxonomy for Accessible Software Engineering

We model an accessibility-engineering task as a triple $\tau = (x, R, O)$: an input x (a specification, a page, an image in context), requirements R drawn from WCAG 2.2 success criteria [44], and an oracle O that maps a model output to check results. We group tasks into six dimensions $C_1, \ldots, C_6$ that mirror widely used families of general LLM capability, so accessibility results can be read alongside general benchmarks (Table I). Two design choices matter. First, every dimension has an oracle whose validity can be checked: deterministic checks where WCAG allows them, seeded faults where ground truth is needed, and a human-calibrated jury where only judgment will do. Second, C_3 and C_5 are measured *within* a session rather than per prompt, because a model that is accessible on the first request but not the twentieth is not accessible in practice.

IV. Evaluation Framework

A. Adherence and the Composite Capability Score

All scores share one oracle convention. For an artifact x (a page, or the part of it a request changed), let $Q(x)$ be its applicable checks: each axe-core rule [45] restricted to WCAG 2.x A/AA that evaluated at least one element in scope (passing if no element violates it), and each applicable keyboard or semantics probe. Adherence is

$$a(x) = \frac{|\{q \in Q(x) : q \text{ passes}\}|}{|Q(x)|}. \tag{1}$$

Counting rules rather than elements keeps thousands of passing text nodes from masking a single failing control. Accessibility barriers are not compensatory: a page with perfect markup but meaningless alt text still excludes blind readers. We therefore aggregate the dimension scores with a weighted geometric mean, which rewards balanced profiles:

$$\mathrm{CCS}_m = \exp\Big(\textstyle\sum_{k=1}^{6} w_k \ln \max(S_{m,k}, \epsilon)\Big), \tag{2}$$

with $w_k = 1/6$ and $\epsilon = 0.01$; the arithmetic mean serves as a sensitivity check.

B. Dynamic Contextual Drift Index (DCDI)

Let model m build a product over T requests in run r. After request t we apply its checks within its scope, obtaining in-session adherence $a_{m,r,t}$. To separate drift from task difficulty and from the state of the code base, we restore the workspace snapshot $S_{m,r,t-1}$ and replay request t in a fresh session with the same system prompt but no history, obtaining $a^0_{m,r,t}$. The two arms differ only in conversational history. We define

$$\mathrm{DCDI}_m = \frac{\sum_r \sum_{t \in \mathcal{T}} \big(a^0_{m,r,t} - a_{m,r,t}\big)}{\sum_r \sum_{t \in \mathcal{T}} a^0_{m,r,t}}, \tag{3}$$

the share of fresh-context adherence lost in-session over the requests $\mathcal{T}$ that both arms implement. DCDI is 0 without drift, 1 if all adherence is lost, and negative if history helps. Because DCDI averages over a session, we also estimate a drift rate β_m, the adherence lost per doubling of context:

$$a^0_{m,r,t} - a_{m,r,t} = \alpha_t + \beta_m \log_2(\Lambda_{m,r,t}/\Lambda_0) + \varepsilon_{m,r,t}, \tag{4}$$

where Λ is the input length in tokens, $\Lambda_0 = 8{,}192$, and α_t are request fixed effects. Finally, a *padded* control replaces the history with irrelevant agent history of the same length, giving a^π and an additive decomposition at checkpoint requests:

$$\mathrm{DCDI} = \underbrace{\frac{\sum(a^0 - a^\pi)}{\sum a^0}}_{\text{length}} + \underbrace{\frac{\sum(a^\pi - a)}{\sum a^0}}_{\text{history}}. \tag{5}$$

TABLE I
SIX CAPABILITY DIMENSIONS INSTANTIATED FOR ACCESSIBLE SOFTWARE ENGINEERING, WITH THE ORACLE AND SCORE USED FOR EACH.

	General capability	Accessibility instantiation	WCAG 2.2 anchors	Oracle	Score S_k
C_1	Formal & symbolic reasoning	Conformance auditing: find failures and map them to success criteria	1.1.1, 1.3.1, 1.4.3, 2.1.1, 2.4.2–2.4.7, 3.1.1, 4.1.2	192 seeded faults with ground truth	Micro-F1 (E1)
C_2	Program synthesis & code engineering	Accessible-by-construction UI code; repair without regressions	as C_1, plus 1.2.2, 2.2.2, 3.3.1, 4.1.3	axe-core rules, keyboard probes, preservation checks	$\frac{1}{2}(S_{\text{gen}} + S_{\text{rep}})$ (E1)
C_3	Autonomous agentic execution	Keeping requirements across a multi-page, tool-using build	all of the above	Step-scoped checks	Mean in-session adherence (E2)
C_4	Cross-modal synthesis	Context-appropriate text alternatives for images	1.1.1	CMBPE jury, BLV-calibrated	Expected win rate (E3)
C_5	Extended-context dynamics	Retaining adherence as the session history grows	all of the above	Fresh and padded controls	$1 - \text{clip}(\text{DCDI}, 0, 1)$ (E2)
C_6	Instruction & safety calibration	Handling requests that conflict with accessibility; honest conformance claims	1.4.1, 1.4.3, 1.4.4, 2.1.1, 2.4.7, 3.3.1, 3.3.2	Pressure-step checks; claim audit	$\frac{1}{2}$(pressure + $1-$overclaim) (E1, E2)

Request order is randomized across runs, so position is not confounded with content.

C. Cross-Model Blind Peer Evaluation (CMBPE)

CMBPE compares outputs from generators $g \in \mathcal{G}$ using one judge j per model family $f(j)$ (Algorithm 1). Outputs are normalized to remove family-identifying formatting and given opaque identifiers; every sampled pair is judged in both orders; and a judge's verdicts on pairs involving its own family are excluded from aggregation (recusal) but kept to measure bias. A verdict $z = 1$ means the first-shown output won, modeled as

$$P(z = 1 \mid j, a, b) = \sigma\big(\theta_a - \theta_b + \delta_j + \gamma_j \, \Delta\ell_{ab}\big), \qquad (6)$$

where θ are quality scores ($\sum_g \theta_g = 0$), δ_j is judge j's first-position bias, γ_j its length bias and $\Delta\ell_{ab}$ the standardized length difference. With θ fixed at the panel estimate, judge j's self-preference η_j is the coefficient of $\mathbf{1}[f(a) = f(j)] - \mathbf{1}[f(b) = f(j)]$ in a refit on all of j's verdicts. A model's capability score is its expected win rate, $S_{m,4} = \frac{1}{|\mathcal{G}|-1} \sum_{g \neq m} \sigma(\theta_m - \theta_g)$. We validate CMBPE three ways: correlation with BLV and crowd ratings, accuracy on pairs whose answer is known by construction, and position consistency.

D. Capability-per-Dollar-Second (CPDS)

A tempting efficiency score divides capability by the logarithm of list price and by token latency. It fails in three ways: $\log_{10}$ of a per-million-token price is zero at \$1 and negative below it, so cheap models receive infinite or negative scores; list prices ignore how many (often hidden reasoning) tokens a model spends per task; and time-to-first-token (seconds) and time-per-output-token (seconds per token) cannot be added. CPDS instead uses the mean billed cost per task, C_m (all attempts, cached and reasoning tokens included), and the median end-to-end latency per task, $L_m = \text{TTFT} + (n_{\text{out}} - 1)\,\text{TPOT}$:

$$\text{CPDS}_m^{(\alpha,\beta)} = \frac{\text{CS}_m}{\big(1 + \log_{10}(1 + \frac{C_m}{c_0})\big)^{\alpha} \big(1 + \log_{10}(1 + \frac{L_m}{\ell_0})\big)^{\beta}}, \qquad (7)$$

Algorithm 1: Cross-Model Blind Peer Evaluation

Input: items $\mathcal{I}$; generators $\mathcal{G}$; judges $\mathcal{J}$, one per family; pairs per item p
Output: quality $\hat{\theta}$; judge biases $\hat{\delta}_j, \hat{\gamma}_j, \hat{\eta}_j$

foreach $i \in \mathcal{I}$, $g \in \mathcal{G}$ **do**
  $\tilde{y}_{g,i} \leftarrow \text{Normalize}(g(i))$ under an opaque id;
$\mathcal{D} \leftarrow \emptyset$;
foreach $i \in \mathcal{I}$ **do**
  $P_i \leftarrow p$ pairs from $\mathcal{G}$, each generator equally often;
  foreach $(a, b) \in P_i$, $j \in \mathcal{J}$, $(u, v) \in \{(a, b), (b, a)\}$ **do**
    $z \leftarrow j(i, \tilde{y}_{u,i}, \tilde{y}_{v,i})$; add $(i, j, u, v, z, \Delta\ell_{uv})$ to $\mathcal{D}$;
$\mathcal{D}^* \leftarrow \{d \in \mathcal{D} : f(j) \notin \{f(u), f(v)\}\}$; ▷ recusal
Fit Eq. (6) on $\mathcal{D}^*$ to get $\hat{\theta}, \hat{\delta}, \hat{\gamma}$;
foreach $j \in \mathcal{J}$ **do**
  fit η_j on j's verdicts in $\mathcal{D}$ with $\theta = \hat{\theta}$;
Bootstrap over items for confidence intervals;

with $c_0 = \$0.01$ and $\ell_0 = 1\,\text{s}$. Both penalties equal 1 at zero cost or latency, grow by about one per decade beyond the reference, and are strictly positive. The exponents encode deployment scenarios: batch remediation ($\alpha = 1, \beta = 0$), an interactive assistant ($\alpha = 0.5, \beta = 1$) and a balanced setting ($\alpha = \beta = 1$). Because any scalar hides trade-offs, we also report the Pareto set over (CS $\uparrow, C \downarrow, L \downarrow$) with each model's membership probability π_m across bootstrap resamples, and test rank stability for $c_0 \in \{0.001, 0.01, 0.1\}$ and $\ell_0 \in \{0.5, 1, 5\}$ s.

V. STUDY DESIGN

Hypotheses. We pre-registered (commit TBD) hypotheses that turn common vendor positioning into testable claims: *H1*, within each vendor the larger model drifts less (lower DCDI) than its smaller sibling; *H2*, the best closed model writes better alt text than the best open-weight model; *H3*, at least one open-weight model is on the batch-repair Pareto frontier ($\pi \geq 0.5$); *H4*, single judges show self-preference and CMBPE beats the median single judge on known-answer pairs; *H5*, a one-sentence request for accessibility improves generated code for most models. Tests are one-sided with Holm correction.

TABLE II

MODELS (OCTOBER 2026). ARCH.: ARCHITECTURE (UND. = UNDISCLOSED). V: IMAGE INPUT. TTC: TEST-TIME REASONING CONTROL (ADAPTIVE = MODEL-DECIDED THINKING STEERED BY AN EFFORT SETTING; EFFORT = SELECTABLE REASONING EFFORT; MODES = THINKING AND NON-THINKING MODES; ALWAYS = THINKING ONLY). PRICE IN USD PER MILLION INPUT/OUTPUT TOKENS. USE: 1 = E1, 2 = E2, G = E3 GENERATOR, J = E3 JUDGE.

Model	Open	Arch.	V	TTC	$ in/out	Use
Claude Opus 5.5	–	und.	✓	adaptive	4/20	1 2 G
Claude Sonnet 5.5	–	und.	✓	adaptive	2/10	1 J
Claude Haiku 5.5	–	und.	✓	adaptive	0.10/0.50[a]	1 2 G
Claude Fable 5.1†	–	und.	✓	adaptive	10/50	1
GPT-6 Sol	–	und.	✓	effort	2/10	1 2 G J
GPT-6 Luna	–	und.	✓	effort	0.10/0.50	1 2 G
GPT-6 Astra†	–	und.	✓	effort	10/50	1
Gemini 4 Argon	–	und.	✓	thinking	2/10[b]	1 2
Gemini 3.8 Flash	–	und.	✓	thinking	0.75/3.75	1 2 G J
DeepSeek V4-Pro	✓	MoE 1.6T/49B	–	modes	1.32/3.96[c]	1 2
Qwen3.8-27B	✓	27B	✓	modes	TBD	1 2 G J
Gemma 4 31B	✓	dense 31B	✓	–	TBD	1 G
Llama 4 Maverick	✓	MoE 400B/17B	✓	–	TBD	1 G
Qwen3.8 2.4T†	✓	MoE 2.4T/95B	–	always	TBD	1

† Optional (budget permitting). [a] Prompts up to 100k tokens. [b] Introductory price until 30 Nov 2026. [c] Peak hours; half price off-peak. Open-weight prices are those of the hosting provider used.

Models. Table II lists the models, chosen as of October 2026 to span vendors, sizes, open and closed weights, dense and mixture-of-experts architectures, and test-time reasoning controls. We use each provider's default temperature and reasoning effort (judges run at the lowest available effort), pin snapshots, and log every setting.

E1: capability. *Generation*: 20 UI tasks drawn from common interface patterns (forms with validation, modal dialogs, tabs, accordions, menus, sortable tables, carousels, comboboxes, switches, sliders, status messages, video, pagination, tooltips, multi-step forms, consent banners) in a fictional public health-clinic service. Prompts never mention accessibility; a *nudged* condition adds one sentence asking for WCAG 2.2 AA. Three samples per task and condition. A follow-up turn asks whether the code conforms; an *overclaim* is a "yes" when our checks find a failure. *Repair* and *audit*: twelve accessible base pages (four adapted from the W3C Before-and-After Demonstration [46], eight written for this study) each yield four faulty variants, and each variant carries four faults seeded by twelve mutation operators (e.g., removed text alternatives, placeholder-only labels, reduced contrast, buttons replaced by `divs`, removed focus indicators, invalid ARIA): 48 faulty pages and 192 faults with ground truth. A repair scores the share of faults resolved minus regressions introduced, and zero if visible text, element ids or DOM structure are not preserved.

E2: drift. Agents build a five-page clinic website over 24 requests using three file tools; the system prompt states the WCAG 2.2 AA requirement once. Four foundation requests come first; the other 20, including six *pressure* requests that conflict with accessibility (e.g., "remove the visible field labels", "disable zooming"), follow in one of three fixed random orders. Each request names a container id so checks can be scoped. Fresh controls run for every request and padded controls at positions 8, 12, 16, 20 and 24. A pressure request scores 1 if the design intent is met accessibly, 0.5 if accessibility is kept but the intent is not met, and 0 otherwise; where no accessible resolution exists, it scores 1 only if accessibility is kept.

E3: judging. We use 68 image descriptions rated in context by BLV participants [41] and 204 rated by crowd workers in ContextRef [42]. Four judges from four families (Claude Sonnet 5.5, GPT-6 Sol, Gemini 3.8 Flash, Qwen3.8-27B) rate every description and compare each original with four degraded versions whose answer is known (an appended irrelevant sentence, repetition, another image's description, truncation). TBD vision-capable generators then write alt text for 120 ContextRef images; 16 of the generator pairs per image are judged in both orders by all judges. A character n-gram classifier tests whether anonymized outputs still reveal their family.

Oracles and analysis. Pages are rendered in Chromium (TBD) via Playwright and checked with axe-core TBD, restricted to WCAG 2.x A/AA rules, plus TBD keyboard and semantics probes (e.g., dialog focus handling, pause controls for moving content, error association), each validated on accessible and inaccessible fixtures. Probes locate widgets by visible text, so any correct implementation can pass. We used native APIs where available (otherwise one gateway with a pinned host), streamed responses to time them, enabled prompt caching, and logged every request, response, token count and price: TBD calls costing TBD between TBD. We report 95% bootstrap confidence intervals (2,000 resamples of tasks, pages, run–request pairs or images) and call a difference only when its interval excludes zero. Methods, prompts and analysis code were frozen before data collection.

VI. RESULTS

A. RQ1: Capability profiles

[Write results: RQ1] Using Table III and values.tex: which models lead each dimension (with CIs); how rankings change across dimensions (Kendall tau between dimension rankings); the default gap (unguided vs. nudged S_{gen}, H5); overclaim rates; repair regressions and full-repair rates; any difference between W3C and authored base pages. Where the data support it, contrast vendor families (Anthropic, OpenAI, Google, open-weight) without generalizing beyond the tested snapshots. About 140 words.

B. RQ2: Drift over long sessions

[Write results: RQ2] Using Fig. 1 and Table III: DCDI and drift rate β per model with CIs; which models drift (CI above 0); the length vs. history decomposition; how many earlier requests lost adherence by the end of the session; mean tool-use cycles per session (`e2-mean-tool-cycles`); pressure-step scores and intent-met rates; H1 after Holm correction. About 150 words.

C. RQ3: Judge reliability

[Write results: RQ3] Using Table IV: compare single judges, the naive panel and CMBPE on BLV and crowd correlation (against the length-only baseline), known-answer accuracy by degradation type, and position consistency; report η_j per judge (H4a) and H4b after Holm; leakage-classifier accuracy vs. majority baseline; whether judges agree more with crowd than BLV ratings. About 140 words.

TABLE III

CAPABILITY PROFILE, DRIFT AND EFFICIENCY. S_1–S_6 AS IN TABLE I; CCS FOR MODELS WITH ALL SIX SCORES; DCDI WITH 95% CI; CPDS FOR BATCH REPAIR (α=1, β=0) AND INTERACTIVE GENERATION (α=0.5, β=1); π: PARETO-MEMBERSHIP PROBABILITY FOR BATCH REPAIR. "–": NOT MEASURED FOR THIS MODEL.

Model	S_1	S_2	S_3	S_4	S_5	S_6	CCS	DCDI [95% CI]	$\text{CPDS}_{\text{batch}}$	$\text{CPDS}_{\text{inter}}$	π_{batch}
Claude Opus 5.5	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Claude Sonnet 5.5	TBD	TBD	–	–	–	–	–	–	TBD	TBD	TBD
Claude Haiku 5.5	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Claude Fable 5.1†	TBD	TBD	–	–	–	–	–	–	TBD	TBD	TBD
GPT-6 Sol	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GPT-6 Luna	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GPT-6 Astra†	TBD	TBD	–	–	–	–	–	–	TBD	TBD	TBD
Gemini 4 Argon	TBD	TBD	TBD	–	TBD	TBD	–	TBD	TBD	TBD	TBD
Gemini 3.8 Flash	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
DeepSeek V4-Pro	TBD	TBD	TBD	–	TBD	TBD	–	TBD	TBD	TBD	TBD
Qwen3.8-27B	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma 4 31B	TBD	TBD	–	TBD	–	–	–	–	TBD	TBD	TBD
Llama 4 Maverick	TBD	TBD	–	TBD	–	–	–	–	TBD	TBD	TBD
Qwen3.8 2.4T†	TBD	TBD	–	–	–	–	–	–	TBD	TBD	TBD



Fig. 1. Accessibility adherence by position in the session (E2), in-session (solid) versus fresh-context control (dashed), with 95% bootstrap bands. The gap between the lines is the drift that DCDI summarizes.

TABLE IV

JUDGE RELIABILITY (E3). r: PEARSON CORRELATION WITH THE MEAN HUMAN RATING (BLV READERS; CROWD WORKERS). ACC: ACCURACY ON KNOWN-ANSWER PAIRS. PC: POSITION CONSISTENCY. γ: LENGTH BIAS; η: SELF-PREFERENCE (LOGITS; SINGLE JUDGES ONLY).

Method	r_{BLV}	r_{crowd}	Acc	PC	γ	η
Claude Sonnet 5.5 (single)	TBD	TBD	TBD	TBD	TBD	TBD
GPT-6 Sol (single)	TBD	TBD	TBD	TBD	TBD	TBD
Gemini 3.8 Flash (single)	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3.8-27B (single)	TBD	TBD	TBD	TBD	TBD	TBD
Naive panel	TBD	TBD	TBD	TBD	TBD	–
CMBPE	TBD	TBD	TBD	–	–	–
Length only (baseline)	TBD	TBD	–	–	–	–



Fig. 2. Repair capability versus mean cost per page (log scale) for batch repair. Marker area shows median latency; filled markers are open-weight models; the line connects Pareto-efficient models.

D. RQ4: Efficiency and affordability

[Write results: RQ4] Using Fig. 2 and Table III: the batch-repair Pareto set with π; CPDS rankings per scenario and their stability across c_0 and ℓ_0 (Kendall tau); cost to repair 1,000 benchmark pages for the cheapest Pareto model vs. the most capable model; H3. About 110 words.

VII. DISCUSSION

Evaluate sessions, not prompts. DCDI's fresh-context control costs one extra short session per request, so it can run as a regression gate for coding assistants: a release should keep late-session adherence close to fresh-context adherence. [TODO: One sentence connecting to the RQ2 finding.]

Recuse, permute and calibrate judges. Whenever LLMs judge accessibility content, the judge panel should exclude the generator's family, present both orders, and report agreement with ratings from disabled people rather than with proxy raters alone. [TODO: One sentence connecting to the RQ3 finding.]

Affordability is an accessibility issue. Organizations that most need low-cost remediation, such as small public bodies and non-profits, are those least able to pay frontier prices. Reporting cost per task and Pareto membership lets them choose with evidence. [TODO: One sentence connecting to the RQ4 finding.]

TABLE V
PRE-SPECIFIED MODEL-SELECTION RULES BY DEPLOYMENT CONTEXT, APPLIED TO OUR RESULTS (CHOICE, THEN RUNNER-UP).

Context	Rule: constraints → objective	Choice (runner-up)
Enterprise compliance	All six scores; $S_5 \geq$ median → max CCS	TBD (TBD)
Research	Open weights → max CCS over measured dimensions	TBD (TBD)
Public sector, NGOs	$\pi_{\text{batch}} \geq 0.5$; $S_2 \geq$ median → max $\text{CPDS}_{\text{batch}}$	TBD (TBD)
Edge, local tools	Open weights, ≤32B parameters → max CCS over measured dimensions	TBD (TBD)

Decision matrix. Table V applies selection rules fixed before data collection to our results; the rules, not our judgment, pick the models.

Threats to validity. *Construct:* automated checks cover only part of WCAG [17], so our adherence scores are upper bounds on conformance and overclaim rates are lower bounds; probes test keyboard operability and exposed semantics, not the full screen-reader experience; DCDI depends on our 24-request script and a single product; and LLM judges approximate, but do not replace, disabled users, while our BLV set is small (68 descriptions) and ContextRef's raters are crowd workers rather than BLV readers. *Internal:* results depend on prompt wording and default settings, which we fix and release; nondeterminism is addressed with three samples or runs; providers may change models behind an identifier, so we log dates and returned model names; fresh-control agents re-read files, which refreshes code in context, and we count this as part of drift; eight base pages were written with LLM assistance, so we report repair results by page provenance; and an LLM from one evaluated family assisted in drafting this paper and the harness, which we mitigate with frozen pre-registered analysis, fully automated scoring and family recusal. *External:* static English web pages in one domain; no component frameworks or native mobile UIs; a snapshot of late-2026 models. *Conclusion:* many comparisons, so we rely on intervals and Holm-corrected hypothesis tests. [TODO: Report each deviation from the pre-registered plan (DEVIATIONS.md), or state that there were none.]

Future work. Procedurally generated tasks and mutants with a private held-out split would reduce contamination and allow dynamic task synthesis; matched reasoning budgets would standardize test-time compute across vendors; and participatory validation with screen-reader users and people with cognitive and motor disabilities, extended to mobile UIs and WCAG 3.0 outcomes, would ground the scores in lived experience.

VIII. CONCLUSION

Static leaderboards cannot tell a team whether a model will keep a site accessible on its twentieth request, whether the judge scoring it is fair, or whether a small public body can afford it. DCDI, CMBPE and CPDS answer these three questions, and our study shows [TODO: one-sentence summary of the main findings]. We release the framework so that choosing models for accessibility work can rest on evidence rather than on leaderboard position.

DATA AVAILABILITY

Tasks, prompts, corpora, harness, analysis code and call logs: [TODO: GitHub URL and Zenodo DOI].

ACKNOWLEDGMENT

Use of generative AI. Following IEEE policy [47]: Claude Opus 5.5 (Anthropic) was used to draft the text of all sections and the experiment specification, prompts and task files; Claude Code was used to implement and run the evaluation harness and analysis and to insert computed results into the text. The authors reviewed, edited and verified all content and take full responsibility for it. The LLMs evaluated in Sections V–VI are objects of study; none of their outputs appear as text of this paper. [TODO: Edit this statement so it matches exactly how AI was used.]

REFERENCES

[1] WebAIM, "The WebAIM million: The 2026 report on the accessibility of the top 1,000,000 home pages," https://webaim.org/projects/million/, 2026.
[2] European Parliament and Council of the European Union, "Directive (EU) 2019/882 on the accessibility requirements for products and services (European Accessibility Act)," Official Journal of the European Union, L 151, 2019.
[3] W. Aljedaani, A. Habib, A. Aljohani, M. Eler, and Y. Feng, "Does ChatGPT generate accessible code? investigating accessibility challenges in LLM-generated source code," in *Proc. 21st Int. Web for All Conf. (W4A)*, 2024.
[4] A.-E. Gurita and R.-D. Vatavu, "When LLM-generated code perpetuates user interface accessibility barriers, how can we break the cycle?" in *Proc. 22nd Int. Web for All Conf. (W4A)*, 2025.
[5] D. M. F. Rabelo, R. Souza, I. Santos, P. Henrique, K. Gama, and W. Viana, "Breaking barriers in mobile accessibility: A study of LLM-generated native Android interfaces," in *Proc. IEEE/ACM 12th Int. Conf. Mobile Software Engineering and Systems (MOBILESoft)*, 2025.
[6] C. Kreer, "WebAccessBench: Digital accessibility reliability in LLM-generated web interfaces," White paper, https://conesible.de/wab, Feb. 2026.
[7] N. Fathallah, D. Hernández, and S. Staab, "AccessGuru: Leveraging LLMs to detect and correct web accessibility violations in HTML code," in *Proc. 27th Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2025.
[8] O. Oyelayo, G. Abushaqra, P. Asadi, D. Dey, and D. E. Costa, "LLM based web accessibility repair: An empirical study of detection, remediation, and cost," arXiv:2605.27716, 2026.
[9] A. G. Mohanbabu and A. Pavel, "Context-aware image descriptions for web accessibility," in *Proc. 26th Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2024.
[10] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos *et al.*, "Chatbot arena: An open platform for evaluating LLMs by human preference," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.
[11] Y. Wang *et al.*, "MMLU-Pro: A more robust and challenging multi-task language understanding benchmark," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
[12] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
[14] H. Suh, M. Tafreshipour, S. Malek, and I. Ahmed, "Human or LLM? a comparative study on accessible code generation capability," arXiv:2503.15885, 2025.
[15] P. Laban, H. Hayashi, Y. Zhou, and J. Neville, "LLMs get lost in multi-turn conversation," arXiv:2505.06120, 2025.

[16] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[17] M. Vigo, J. Brown, and V. Conway, "Benchmarking web accessibility evaluation tools: Measuring the harm of sole reliance on automated tests," in *Proc. 10th Int. Cross-Disciplinary Conf. Web Accessibility (W4A)*, 2013.

[18] T. Calò, A.-E. Gurita, and L. De Russis, "Measuring the semantic accessibility gap in LLM-generated web UIs," in *Extended Abstracts of the CHI Conf. Human Factors in Computing Systems (CHI EA)*, 2026.

[19] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] P. Wang *et al.*, "Large language models are not fair evaluators," in *Proc. 62nd Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[21] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. the method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.

[23] P. Liang *et al.*, "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023.

[24] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. L. de Lacalle, and E. Agirre, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[25] C.-P. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" in *Proc. Conf. Language Modeling (COLM)*, 2024.

[26] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," arXiv:2406.12045, 2024.

[27] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[28] P. Verga *et al.*, "Replacing judges with juries: Evaluating LLM generations with a panel of diverse models," arXiv:2404.18796, 2024.

[29] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, "Length-controlled AlpacaEval: A simple way to debias automatic evaluators," arXiv:2404.04475, 2024.

[30] T. Ahmed, P. Devanbu, C. Treude, and M. Pradel, "Can LLMs replace manual annotation of software engineering artifacts?" in *Proc. 22nd Int. Conf. Mining Software Repositories (MSR)*, 2025.

[31] S. Baltes *et al.*, "Guidelines for empirical studies in software engineering involving large language models," arXiv:2508.15503, 2025.

[32] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: The threats of using LLMs in software engineering," in *Proc. 46th Int. Conf. Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024.

[33] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," *Transactions on Machine Learning Research*, 2025.

[34] M. H. Erol, B. El, M. Suzgun, M. Yuksekgonul, and J. Zou, "Cost-of-pass: An economic framework for evaluating language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026.

[35] A. Alshayban, I. Ahmed, and S. Malek, "Accessibility issues in Android apps: State of affairs, sentiments, and ways forward," in *Proc. 42nd Int. Conf. Software Engineering (ICSE)*, 2020.

[36] J. Chen, C. Chen, Z. Xing, X. Xu, L. Zhu, G. Li, and J. Wang, "Unblind your apps: Predicting natural-language labels for mobile GUI components by deep learning," in *Proc. 42nd Int. Conf. Software Engineering (ICSE)*, 2020.

[37] T. Bi, X. Xia, D. Lo, J. Grundy, T. Zimmermann, and D. Ford, "Accessibility in software practice: A practitioner's perspective," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 4, 2022.

[38] P. Mowar, Y.-H. Peng, J. Wu, A. Steinfeld, and J. P. Bigham, "CodeA11y: Making AI coding assistants useful for accessible web development," arXiv:2502.10884, 2025.

[39] L. B. Wanscher, M. H. Lorensen, M. A. Shafiq, M. T. Moghaddam, and M. Alipour, "From blind edits to verified repair: Building trustworthy user-side LLM agents for web accessibility," in *Companion Proc. 28th Int. Conf. Multimodal Interaction (ICMI)*, 2026, arXiv:2608.24913.

[40] A. Stangl, N. Verma, K. R. Fleischmann, M. R. Morris, and D. Gurari, "Going beyond one-size-fits-all image descriptions to satisfy the information wants of people who are blind or have low vision," in *Proc. 23rd Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2021.

[41] E. Kreiss, C. Bennett, S. Hooshmand, E. Zelikman, M. R. Morris, and C. Potts, "Context matters for image descriptions for accessibility: Challenges for referenceless evaluation metrics," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2022, pp. 4685–4697.

[42] E. Kreiss, E. Zelikman, C. Potts, and N. Haber, "ContextRef: Evaluating referenceless metrics for image description generation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[43] W. Aljedaani and R. H. Mollik, "Large language models for web accessibility: A systematic literature review," in *Proc. 23rd Int. Web for All Conf. (W4A)*, 2026, arXiv:2605.13873.

[44] W3C, "Web content accessibility guidelines (WCAG) 2.2," W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[45] Deque Systems, "axe-core: Accessibility engine for automated web UI testing," https://github.com/dequelabs/axe-core, accessed Oct. 2026.

[46] W3C Web Accessibility Initiative, "Before and after demonstration (BAD)," https://www.w3.org/WAI/demos/bad/, accessed Oct. 2026.

[47] IEEE, "Author guidelines for artificial intelligence (AI)-generated text," https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/, accessed Oct. 2026.